Advanced Offline Detection & Fault-Tolerant System

Three-State Machine with Flash Caching & Exponential Backoff Sync

Institution: Padmabhooshan Vasantraodada Patil Institute of Technology (PVPIT), Budhgaon, Sangli

Project Track: Fault-Tolerant Systems & Non-Volatile Storage Architectures

Document: Technical Specification & System Report

---

1. System Architecture Diagram

The system is built around a centralized state supervisor loop that continuously tracks local connectivity parameters. When network interruptions occur, the system safely reroutes standard data messages to the local SPIFFS non-volatile flash storage layer, allowing core sensing tasks to continue running without interruptions or data loss.

2. FreeRTOS Task Structure Explanation

To implement real-time fault detection and data preservation without stalling the processor, execution paths are handled by dedicated FreeRTOS tasks configured with strict execution priorities.

Task Priority and Scheduling Parameters

Task Handle Name	Priority	Execution Strategy	Functional System Objective
vTaskNetworkCheck	Highest (5)	500ms (Periodic)	Pings local sockets, monitors Wi-Fi RSSI, checks MQTT connection flags, and updates the core State Machine.
vTaskSensorSample	High (4)	100ms (Periodic)	Polls active patient metrics; forwards data packets directly to cloud queues or local storage tasks.
vTaskFlashCache	Medium (3)	Event-Driven	Handles flash memory file I/O operations, saving data packets to local SPIFFS blocks during network drops.
vTaskLocalUI	Low (1)	250ms (Periodic)	Drives the local OLED display layouts, rendering active system status strings and network health parameters.
vTaskWatchdog	Highest (6)	100ms (Periodic)	Independent execution supervisor; resets the system into a safe fallback mode if a storage lockup occurs.

Inter-Task Communication Primitives

- **System Status Enumeration Mutex:** Encapsulates the current active connectivity state variable (ONLINE, DEGRADED, OFFLINE), protecting it from multi-task write corruption.
- **Data Synchronization Queue:** A thread-safe data buffer used to queue data points waiting for storage or flash data flushing.

3. System Feed Mapping Table

The network interface utilizes separated MQTT topics to isolate real-time metrics, diagnostic reporting channels, and system alerts.

MQTT Topic Namespace	Data Format	Direction	Description / Operational Context
`/system/network/state`	String ID	Publish	Reports current operational status flags: `"ONLINE"`, `"DEGRADED"`, or `"OFFLINE"`.
`/system/data/sync`	JSON Array	Publish	Flushes accumulated local historical records once connection with the cloud is restored.
`/system/telemetry/vitals`	JSON String	Publish	Real-time patient diagnostics stream (active when network state is ONLINE).
`/system/diagnostics/rssi`	Integer (dBm)	Publish	Monitors Wi-Fi signal strengths to provide early warnings for network degradation.

4. Remote UI Diagnostics Dashboard Design

The remote tracking dashboard provides clear visual elements to log connection states and track historical sync updates.

[Centralized Network Operations Management Dashboard View]

• Connection Status Light: Multi-color indicator (Green = Connected, Amber = Degraded Link, Grey/ Red = Disconnected).

• Local Cache Counter Widget: Progress tracker showing total unsynced data records stored in flash memory.

• Synchronization Timeline Graph: Visual data chart displaying live real-time entries alongside recovered historical records.

5. Three-State Connectivity Engine Core Logic

The core firmware processes connection stability parameters through a structured state machine categorized into three distinct operational modes:

- **ONLINE State:** Validates successful Wi-Fi authentication and active MQTT broker authorization. Data streams straight to network topics, and the OLED panel shows connection health status info.
- **DEGRADED State:** Triggered when Wi-Fi remains connected but MQTT ping confirmations time out or fail. The device continues trying to re-establish connection while storing data to help diagnose connection drops.
- **OFFLINE State:** Triggered when the physical Wi-Fi connection fails completely. The system instantly switches to local storage mode, changes the local OLED display layout to **"LOGGING OFFLINE"**, and activates distinct buzzer alert audio codes to notify on-site staff.

6. Buzzer Diagnostic Audio Alerts Coding

To help maintenance staff quickly troubleshoot issues without needing to check cloud logs, the local alarm system plays distinct audio frequencies depending on the exact point of failure:

Detected Hardware Fault	Buzzer Frequency	Pulse Pattern Specification	Intended On-Site Diagnostic Meaning
Wi-Fi Disconnect	1200 Hz	Single Long Tone (1.5s On, 1s Off)	Physical wireless connection dropped or local router power loss.
MQTT Handshake Fail	2400 Hz	Double Short Pip (200ms Pulse, 200ms Gap)	Local network active, but connection to cloud broker failed.
SPIFFS Write Exception	3500 Hz	Continuous Rapid Chirp (50ms On/Off)	Local storage full or flash memory access error; risk of data loss.

7. Local Flash File System Storage Implementation

During extended network drops, data packets are serialized into JSON text formats and saved directly into the local SPIFFS flash memory cluster. This method protects data from power losses or accidental restarts:

```
// Local SPIFFS Data Logging Sequence Block
void cache_vitals_to_flash(PatientData_t data) {
    // Open historical data log file in append mode
    File logFile = SPIFFS.open("/vitals_cache.json", "a");
    if (!logFile) {
        trigger_diagnostic_buzzer(SPIFFS_FAULT);
        return;
    }

    // Serialize data structured into a clean JSON string line entries
    logFile.printf("{\"t\":%u,\"h\":%d,\"s\":%d}\n",
                   data.timestamp, data.heartRate, data.spo2);
    logFile.close();
}
```

8. Exponential Backoff Synchronization Recovery

To prevent the device from overloading the network or web servers with continuous rapid reconnection attempts after a long outage, recovery routines apply an **Exponential Backoff with Jitter** retry algorithm.

The time interval between retry attempts increases exponentially with each consecutive failure, up to a maximum safety ceiling limit ($T_{max} = 300 ext{ seconds}$):

$$T_{retry} = \min\left(T_{max}, \; T_{base} \cdot 2^{ ext{attempt}} + ext{random_jitter} ight)$$

Where $T_{base} = 2 ext{ seconds}$. Adding a small random jitter prevents multiple devices on the same network from attempting to reconnect at the exact same moment, which could cause a network collision.

Once connection stability is confirmed, the recovery task opens the `/vitals_cache.json` file, parses the accumulated records line by line, and uploads them to the cloud. After a successful sync operation is verified, the cache file is safely cleared to free up flash memory space.